



WRO FUTUROS INGENIEROS 2026

Journal

NPC

“Proyecto Mahoraga”

COLEGIO JEFFERSON
ESTADO MIRANDA
VENEZUELA

MIEMBROS
LEONARDO CARRASQUERO -16 AÑOS
JUAN IGNACIO GONZALEZ - 16 AÑOS
VU TUE ANH - 16 AÑOS

COACH
ALFREDO GONZALEZ



PORTADA

TABLA DE CONTENIDOS

1. INTRODUCCIÓN

- 1.1 Nuestro Equipo**
- 1.2 Nuestro Espacio de Trabajo**
- 1.3 Experiencia Previa (WRO 2025)**
- 1.4 Objetivos para WRO 2026**

2. CRONOLOGÍA DEL PROYECTO

- 2.1 Fase 1: Investigación y Planificación**
- 2.2 Fase 2: Diseño Conceptual**
- 2.3 Fase 3: Prototipado**
- 2.4 Fase 4: Pruebas e Iteración**

1. INTRODUCCIÓN

Este documento presenta el proceso de ingeniería del equipo NPC (Non Player Character), para la competencia WRO Futuros Ingenieros 2026.

La idea es mostrar un Journal o Diario de nuestro proceso de avance en el desarrollo de nuestro robot **Proyecto Mahoraga** que sirva a otros para poder seguir nuestros pasos.

1.1 Nuestro Equipo



Este es nuestro segundo año en competencias de la WRO, con el objetivo de lograr un vehículo verdaderamente autónomo que supere con éxito los retos de la competencia WRO Futuros Ingenieros 2026 a Nivel Nacional y nos permita obtener nuestro cupo en la Final Internacional de Puerto Rico.

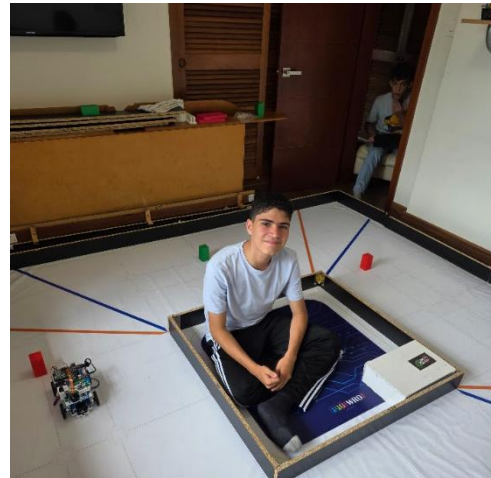
Vu Tue Anh, compitió el año pasado en la categoría de Misiones Roboticas, mientras que Leonardo y Juan participaron en la categoría de Futuros Ingenieros, llegando a la Final Nacional de Venezuela donde obtuvimos el 5to Lugar, mientras que en un invitational en Weifang, China logramos el 1er Lugar.

1.2 Nuestro Espacio de Trabajo

Trabajamos en el Colegio Jefferson todos los viernes y organizamos reuniones en casa de Leonardo semanalmente. Además, avanzábamos de manera independiente durante la semana.

Contamos con una pista con las condiciones y medidas originales de la WRO para hacer pruebas y ajustes.

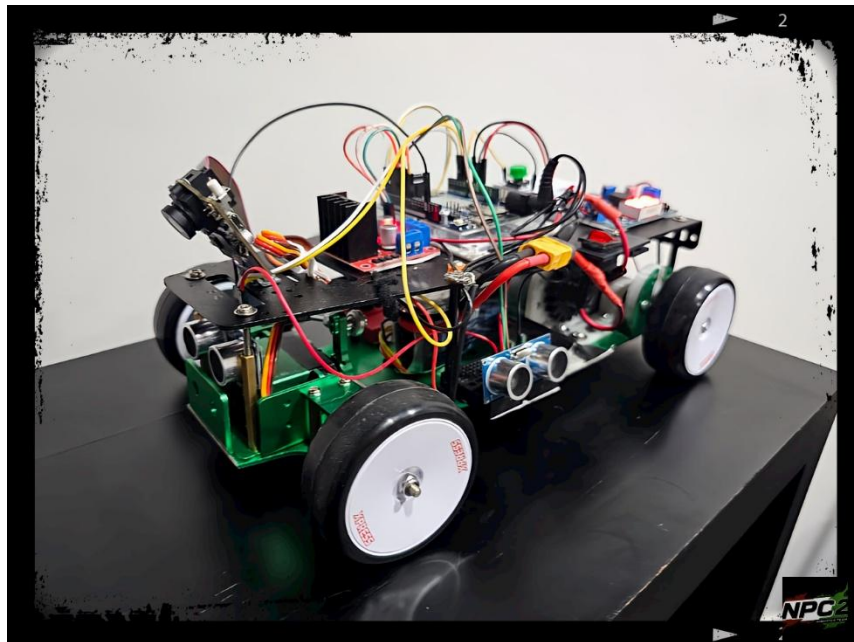
Además, tenemos diversas herramientas e inventario de piezas y partes: Impresora 3D BambuLab P2S, equipo de soldadura, multímetro, destornilladores y herramientas en general. Piezas eléctricas y mecánicas compradas de repuesto, así como multiplicidad de cables, termoencogibles.





1.3 Experiencia Previa (WRO 2025)

Este es nuestro segundo año en competencias de la WRO, el año pasado participamos con “Crazy Diamond”, obteniendo una calificación destacada en los 3 eventos regionales que participamos y logrando la 2da puntuación más alta de toda la ronda clasificatoria a la Final Nacional de Venezuela, donde obtuvimos el 5to Lugar.



1.4 Objetivos para WRO 2026

La competencia 2025 fue nuestra primera experiencia, aprendimos muchísimo y el reto era mejorar para el 2026 y alcanzar la Final Internacional.

Aprendimos que:

Nuestra primera experiencia en la competencia de 2025 fue un gran aprendizaje. Aunque el panorama fue retador, nos sirvió para darnos cuenta de lo que realmente necesitábamos para subir de nivel y ponernos como meta clara alcanzar la Final Internacional este 2026.

Estas fueron las lecciones clave que nos dejaron esa primera participación:

1. El tiempo es clave: Entendimos que no podíamos perder ni un día. En la temporada pasada empezamos a trabajar en abril, pero para este ciclo nos propusimos arrancar en enero, apenas la WRO oficializó el reto y sus nuevas reglas. Esos tres meses de ventaja hicieron toda la diferencia para planificar con calma.
2. Evolución en hardware y software: Ver de cerca las soluciones que llevaron los equipos a la Final Internacional 2025 nos abrió los ojos. Nos dimos cuenta de que, si queríamos ser competitivos, nuestro robot actual no iba a bastar; necesitábamos un salto de calidad drástico tanto en los componentes físicos como en la arquitectura del código.
3. Aprender de la comunidad: Pasamos mucho tiempo investigando, analizando videos de competencias previas y, sobre todo, estudiando a fondo repositorios de GitHub de equipos exitosos en 2024 y 2025. Gracias a eso, pudimos estructurar una hoja de ruta súper clara con los cambios técnicos exactos que Proyecto Mahoraga necesitaba para este 2026.

Basándonos en nuestra experiencia de 2025, establecimos objetivos para 2026:

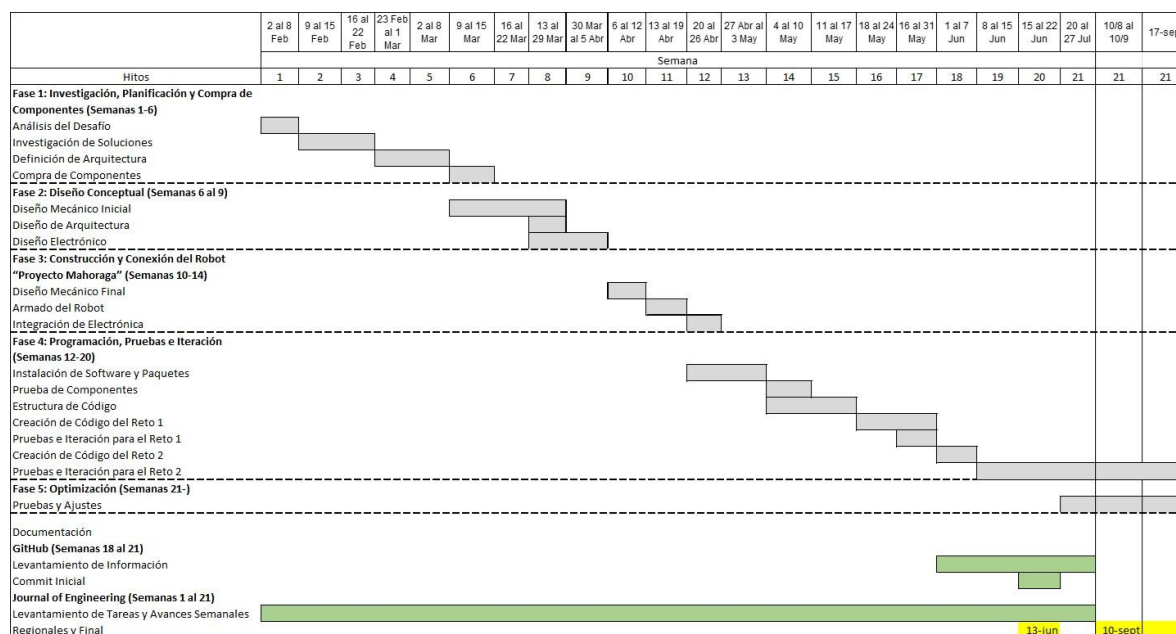
Objetivos Técnicos

- 1) Tener un robot más pequeño y ágil para tener mayor margen en la evasión de obstáculos.
- 2) Tener un chasis hecho a la medida de las necesidades y no uno comercial como empleamos en 2025.
- 3) Incorporar un LiDAR para un mapeo robusto de obstáculos y bordes
- 4) Incorporar un IMU para tener un desarrollo mas robusto que nos permitiera medir de una mejor manera los ángulos de giro y las desviaciones
- 5) Un microprocesador mas potente para poder tener una fluidez en la transmisión de datos de los sensores y motores.
- 6) Incorporar un procesador de alta gama como el Raspberry Pi 5 de 16 Gb para garantizar el manejo de las imágenes de la cámara y el LiDAR, al tiempo de centralizar el control del robot.



Herramientas de Gestión

- Comunicación diaria: Mantuvimos un canal activo y constante a través de un grupo de WhatsApp entre nosotros y nuestro coach. Esto nos permitió resolver dudas rápidas, coordinar las sesiones de trabajo y reportar cualquier imprevisto técnico al momento.
- Documentación en la nube: Centralizamos toda nuestra información en Google Drive. Ahí organizamos desde las carpetas de investigación inicial, los avances del código, el registro fotográfico de las pruebas y los resúmenes de cada sesión para no perder el hilo de lo que hacíamos. Todo esto nos sirvió como base para estructurar de manera sólida nuestro Journal.



2.1 Fase 1: Investigación, Planificación y Compra de Componentes (Semanas 1-6)

	2 al 8 Feb	9 al 15 Feb	16 al 22 Feb	23 Feb al 1 Mar	2 al 8 Mar	9 al 15 Mar	16 al 22 Mar
	Semana						
Hitos	1	2	3	4	5	6	7
Fase 1: Investigación, Planificación y Compra de Componentes (Semanas 1-6)							
Análisis del Desafío							
Investigación de Soluciones							
Definición de Arquitectura							
Compra de Componentes							

Semana 1: Análisis del Desafío

Comenzamos con una sesión de trabajo leyendo detalladamente las reglas de WRO 2026. Identificamos los cambios respecto a 2025 y como esto podía afectar.

Cambio 1: Reto 2 partiendo del Estacionamiento: Se aclara en las reglas 2026 que los puntos de partida desde el estacionamiento en el reto 2 solo se adjudicarán si el equipo cumple una vuelta efectiva.

Efecto: En 2025, usamos la estrategia de salir desde el estacionamiento y obtuvimos la puntuación pese a no completar 1 vuelta, lo cual sería algo a considerar si decidimos partir del estacionamiento en 2026.

Cambio 2: Se promueve el uso de regla sorpresa, que pese no estar en las normas 2025, no se implementó al menos en Venezuela.

Efecto: Importancia de tener un código lo más simple posible y con anotaciones para identificar ajustes necesarios según el cambio de condiciones.

Cambio 3: Normas y elementos del Github mucho más específicos y explicativos.

Efecto: En 2025 nuestra puntuación del Github tenía importantes posibilidades de mejora y acordamos que eso podría hacer una diferencia sustancial en el 2026.

Adicionalmente, repasamos estrategias a un alto nivel de cómo superar este año los retos, con mayor énfasis en el reto 2 que fue el que más dificultad nos dio en 2025.

Semana 2 y 3: Investigación de Soluciones

Si bien esto lo abordamos en la documentación de **“Selección de Componentes y Comparación con 2025”** creemos que al ser parte importante de nuestras decisiones lo repetimos acá.

Investigamos soluciones de los equipos con los mejores puntajes en la Final Internacional 2025. Hicimos cuadros comparativos de los componentes que utilizaron, así como el tipo de lenguaje de programación.

Analizamos y sacamos conclusiones de la información en los repositorios y los videos en Youtube.

Equipo	Partidos clasificatorios		Partidos finales		Documentación	Σ	Github
1. 6030 - Double-X	30.0 (30)	30.0 (37)	62.0 (63)	62.0 (49)	29.3	121.3 (79)	https://github.com/ejm22/WRO-FE-XX
2. 6018 - KMIDS GFM	30.0 (43)	30.0 (43)	62.0 (60)	62.0 (63)	29.0	121 (103)	https://github.com/Chayanon-Ninyawee/KMIDS-GFM-Future-Engineer-2025
3. 6001 - STORMS NGR	30.0 (24)	30.0 (22)	62.0 (87)	62.0 (69)	29.0	121 (91)	https://github.com/MoCsabi/WRO2025-FE-StormsNGR
4. 6009 - FireOnAllCylinders	30.0 (41)	30.0 (44)	47.0 (65)	62.0 (73)	27.3	119.3 (114)	https://github.com/kirkhu/WRO2025_FE-Fire-On-All-Cylinders
5. 6003 - ANTI	30.0 (22)	0.0 (180)	62.0 (47)	47.0 (42)	27.0	119 (69)	https://github.com/atakanersoy/WRO2025_FE_ANTI
6. 6045 - Team LazyGo	30.0 (45)	5.0 (14)	29.0 (53)	62.0 (110)	27.0	119 (155)	https://github.com/A-N-M-Noor/LazyGo_WRO2025
7. 6006 - KMW Battlepillars	30.0 (63)	30.0 (64)	62.0 (162)	27.0 (110)	27.0	119 (225)	https://github.com/Battlepillars/Wro
8. 6042 - SQ-391	30.0 (77)	30.0 (99)	62.0 (135)	62.0 (99)	26.0	118 (176)	https://github.com/Alhadi1123/SQ-391_WRO_Future_Engineers_2025
9. 6029 - Ishikawa KOSEN	30.0 (112)	30.0 (124)	62.0 (157)	11.0 (38)	25.7	117.7 (269)	https://github.com/UNOsan118/WRO2025_FE_INCT
10. 6011 - QYZYLORDA Flame	30.0 (63)	30.0 (51)	47.0 (114)	62.0 (120)	25.0	117 (171)	https://github.com/QZOFlameFE/FE2025_1st_repo_ByFlame
11. 6024 - Greenbotics	30.0 (45)	30.0 (52)	62.0 (75)	62.0 (61)	24.0	116 (106)	https://github.com/Devansh-awat/greenbotics
12. 6012 - shahrood RC	30.0 (41)	0.0 (180)	44.0 (120)	62.0 (141)	24.0	116 (182)	https://github.com/ShahroodRC/WRO2025-FE-ShahroodRC
13. 6058 - X-UniTech	30.0 (34)	30.0 (31)	47.0 (76)	62.0 (89)	23.3	115.3 (120)	https://github.com/vunxiuxuan/X-UniTech-IPGKS
14. 6025 - Paraducks	30.0 (66)	30.0 (76)	62.0 (173)	18.0 (46)	21.7	113.7 (239)	https://github.com/DevShah1407/WRO-Paraducks
15. 6052 - AZE 44	30.0 (61)	30.0 (64)	8.0 (27)	62.0 (177)	20.3	112.3 (238)	https://github.com/coolboy19521/AZE44

Lo que aprendimos de otros equipos:

- Solo 3 del Top 15 uso soluciones en Lego EV3, pero además lo descartamos porque queríamos trabajar con soluciones más avanzadas.
- El Raspberry Pi y principalmente la versión 5, era el controlador más utilizado entre los equipos de mayor puntaje.
- El Arduino era usado por solo 2 equipos y solo como microcontrolador acompañante. Evidenciando la desventaja con la que trabajamos en 2025. ESP32 y el Raspberry Pico 2 resultaron como los microcontroladores más usados.
- 6 del top 10, utilizaron un LiDAR y tenían estrategias robustas de detección.
- Prácticamente todos los equipos utilizaron un IMU, versiones BNO085 y BNO055.
- Los equipos utilizaron controladores de motores más modernos y avanzados que el L298N que utilizamos en 2025, recomendado solo para experimentos de robótica simple.
- La mayoría de los equipos usaron chasis y piezas hechas en 3D. Aquellos que usaron chasis comerciales hicieron modificaciones importantes y complementaron con piezas impresas en 3D.

Acá anexamos un cuadro comparativo.

Equipo	Cámara	Sensor de Distancia	Controlador de Motores	Lenguaje de Programación	Controlador Principal	Controlador Secundario	IMU	Motor	Servo	Diferencial	Convertidor de Voltaje	Otro	Largo	Acho
LazyGo	Logitech C270	LIDAR (RPLidar C1)	VNH2SP30 Motor Driver	Python, C++ (ROS2),	Raspberry Pi 5	ESP32 Microcontroller. ESP-WROOM-32 30/38Pin	BNO055 9-Axis IMU	25GA Gear Motors + Encoder	PS1171MG Servo	LEGO Differential (Gen 3)	5V 20A Buck Converter		20	10
SQ-391	Raspberry Pi Camera Module 3	4 Ultrasoníc o (URM09)	DRV8871 Motor Driver	Python (ROS Noetic),	Raspberry Pi 4 Model B	ESP32-WROOM-32E	BNO085	Core DC Motor (6V, 400 RPM)	TowerPro MG996R Servo y Micro Servo Motor SG90		XL4005 DC-DC Step-down	PCA9685 Servo Driver HAT		
Battlepillars	Raspberry Pi Camera Module 3 Wide	LIDAR (RpLidar S2)	Quicrun WP 1080-G2	Python	Raspberry Pi 5	Servo Controller (Adafruit)	SparkFun Optical Tracking Odometry Sensor. Porque el motor no tiene encoder	23-turn brushed 370-size LaTrax® motor with bullet connectors	Traxxas Waterproof Sub-Micro Servo (model number 2065A)		Pololu 5V Step-down		19	12,8
KMIDS-GFM	Fish-Eye Lens RPI 5MP IR Camera	LIDAR (RPLidar S2)	DRV8871 Motor Driver	C++	Raspberry Pi 5	Raspberry Pi Pico 2	BNO085	N20 Motor, 6V 500 RPM	Servo: S0009M	LEGO differential gear system	UPS EP-0136 DC-DC Boost Step-Up	Transistor: N-Channel MOSFET	24,4	12
Ishikawa KOSEN	2DOF Monocular Camera	LIDAR (STL-19P D500)	RRC Lite Controller	Python (ROS 2)	Raspberry Pi 5	RRC Lite Controller	BNO055 9-Axis IMU	DC Geared Motor w/ Encoder	Hiwonder LFD-01 Servo		Hiwonder RRC Lite Controller	Control Board for LIDAR incluido en el kit del Lidar con el RRC Controler	19,5	15
Double-X	Pi Camera 3 Wide	Basado en Visión (Cámara)	DRV8825 (Stepper driver)	Python y C++	Raspberry Pi 5	Arduino Uno R3	No USA	NEMA 17 Stepper Motor	HS 2258B Servo Motor		No especificado		28,5	13,5
Greenbotics	Raspberry Pi Camera Module 3	ToF (VL53L1X/L8CX) y Ultrasoníc o (URM09) y VL53L8CX	TB6612FNG Dual Motor Driver	Python	Raspberry Pi 5		BNO055 IMU	LEGO EV3 Medium Motor,	Servo SG90	LEGO differential gear system	XY-3606	Multiplexor I2C (TCA9548A)		
StormsNGR	Cámara (Modelo no especificad o)	LIDAR sensor STL-19P, D500 developme nt kit	DRV8871-MK	Python y C	Raspberry Pi 5 4GB	Microcontroller (ESP) NodeMCU-32S	BNO085	Hiwonder JGB37-520R30-12	Steering servo Carson CS-3	Carbon fiber Lego technic axl	XL4015-3D-STDN-PSU		27	19,7
AZE44	Raspberry Pi Camera	Yahboom T-mini Plus LIDAR + 1 HC-SR04	TB6612FNG Dual Motor Driver	Python (ROS 2),	Raspberry Pi 5	Raspberry Pi Pico W RP2040	BNO085	DC Motor con Encoder	Servo Motor TD8120MG				19,5	15
Fire-On-All-Cylinders	SONY IMX477 camera module	2 HCSR04 Ultrasoníc o e Infrarrojo	L293d motor controller		Nvidia Jetson Orin Nano	Raspberry Pi Pico W RP2040	BNO055 IMU			Lego 62821b Differential Gear			15,1	9,27
ANTI	GC2145 2MP Camera	ToF (VL53L1X y VL53L3CX)	DRV8833	Python (MicroPython), C++	STM32H747 (Visión)	nRF52832 (Sensores)	LSM6DSOX	Motor DC N20 de 1500 RPM con codificador	servomotor submicro Feetech FS0307		SK1308 Voltage Booster LiPo Rider Plus		11	5,7

Semana 4 y 5: Definición de Arquitectura

Definimos la arquitectura general de nuestro vehículo y comparamos entre distintos tipos de alternativas para cada componente. Se dio mucho peso en aquellos componentes más robustos, pero también en aquellos que ofrecieran mayor compatibilidad con el resto de los componentes.

Las decisiones de arquitectura tomadas se explican con más detalle en “**Selección de Componentes y Comparación con 2025**”, pero resumimos a continuación la lista de componentes.

Categoría	Componente	Modelo / SKU	Cant.	Link
Cómputo	SBC alto nivel	Raspberry Pi 5 — 16 GB RAM	1	Adafruit 6125
	Refrigeración	Official Raspberry Pi 5 Active Cooler	1	Adafruit 5815
	Base NVMe	Pimoroni NVMe Base PIM699	1	Adafruit 5845
	SSD	Silicon Power 128 GB M.2 NVMe Gen3	1	Amazon
	MicroSD (backup)	Silicon Power 32 GB	1	Amazon
	Microcontrolador	Pimoroni Pico Plus 2 (RP2350) PIM724	1	Adafruit 6244
	Adapter terminal	Adafruit Terminal PiCowbell con sockets	1	Adafruit 5907
Sensores	LiDAR 360°	LDROBOT STL-27L (25 m)	1	DFRobot 2726
	Cámara	Raspberry Pi Global Shutter (IMX296)	1	Adafruit 5702
	Lente cámara	CCTV M12 2.8 mm F2.0 1/2.7"	1	Amazon
	Cable cámara	FPC 22-pin → 15-pin, 200 mm (RPI 5)	1	Adafruit 5818
	ToF láser	Adafruit VL53L4CD STEMMA QT	2	Adafruit 5396
	IMU 9-DOF	Adafruit BNO085 STEMMA QT	1	Adafruit 4754
	Concentrador I ² C	SparkFun Qwiic MultiPort	1	SparkFun
Actuadores	Cable Qwiic	Flexible Qwiic Cable 50 mm	4	SparkFun
	Motor + encoder	Pololu 34:1 25Dx67L MP 12V + encoder 48 CPR	1	Pololu 4864
	Driver de motor	Pololu VNH5019 Motor Driver Carrier	1	Pololu 1451
	Servo de dirección	Savox SC-1251MG digital coreless	1	Savox
Potencia	Batería	Zeee LiPo 3S 11.1 V 3200 mAh 50C	1	Zeee Battery
	Regulador 5V — Lógica	Pololu D24V90F5 (5 V, 9 A)	1	Pololu 2866
	Regulador 5V — Servo	Pololu D36V50F5 (5 V, 5.5 A)	1	Pololu 4091
Mecánica	Ruedas	64 mm aluminio + neumático goma	4	Amazon
	Eje trasero	KYX Racing Steel Drive Shaft (Tamiya DT04)	1	Amazon
	Manguetas Ackermann	KYX Racing Aluminum Steering Knuckles	2	Amazon
	Servo saver	Tamiya TT-02 Hi-Torque	1	Tamiya
	Tie rods	M3 ajustables uxcell	2	Amazon
	Ejes	Acero inoxidable 5 mm × 100 mm uxcell	2	Amazon
Cables	USB-C Pi ↔ Pico	SUNGUY USB-C 3.1 Gen2 — 6 in y 1 ft	2	Amazon
Mecánica 3D	Filamento	PLA Bambu Lab	—	Bambu Lab
	Impresora 3D	Bambu Lab P2S	—	Bambu Lab

Semana 6: Compra de Componentes

Una vez que identificamos lo que necesitábamos, y a diferencia de 2025, cuando usamos componentes más básicos, decidimos importar la mayoría de las piezas para conseguir mejores precios y más variedad. Para evitar problemas con los tiempos de envío si algo fallaba, dejamos de lado las partes genéricas y priorizamos marcas reconocidas. Toda la compra la gestionamos a través de Amazon.com y directamente en las páginas oficiales de fabricantes especializados, como Pololu para los motores y componentes electrónicos, y Adafruit para los sensores y microcontroladores

2.2 Fase 2: Diseño Conceptual (Semanas 6-9)

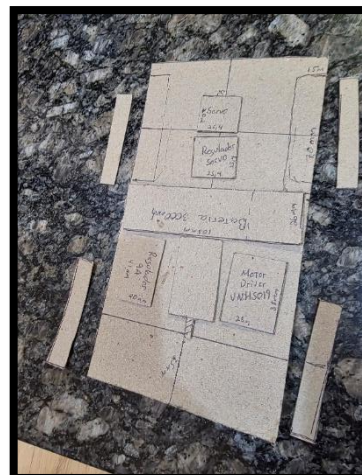
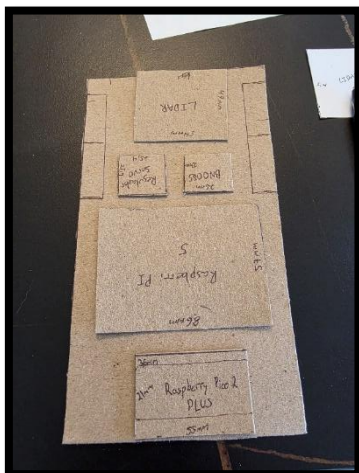
	2 al 8 Feb	9 al 15 Feb	16 al 22 Feb	23 Feb al 1 Mar	2 al 8 Mar	9 al 15 Mar	16 al 22 Mar	23 al 29 Mar	30 Mar al 5 Abr	6 al 12 Abr
	Semana									
Hitos	1	2	3	4	5	6	7	8	9	10
Fase 2: Diseño Conceptual (Semanas 6 al 9)										
Diseño Mecánico Inicial										
Diseño de Arquitectura										
Diseño Electrónico										

Semana 6 a 8: Diseño Mecánico Inicial

Con la compra de los componentes y en espera de su llegada, empezamos a avanzar en la definición de cómo podríamos distribuir los componentes elegidos en un espacio físico lo más limitado posible para minimizar el tamaño del vehículo, uno de los principales objetivos que nos habíamos puesto.

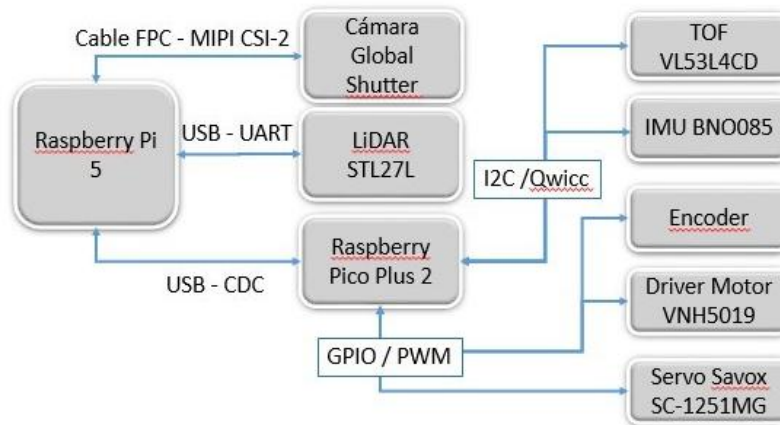
Utilizamos pedazos de cartón con el tamaño que obtuvimos de las características de cada componente en su página web y definimos un tamaño de chasis y distribución ideal.

Este proceso nos llevó a varias iteraciones, que se fue terminando de definir una vez que llegaron los componentes y que incluso llevó a al rediseño en varias oportunidades del chasis como se comenta en la siguiente Fase 3.



Semana 8: Diseño de Arquitectura

Definimos la arquitectura principal, que iba a manejar el Raspberry Pi 5 y que iba a manejar el Raspberry Pico 2 para garantizar la mayor eficiencia y la menor complejidad de integración.

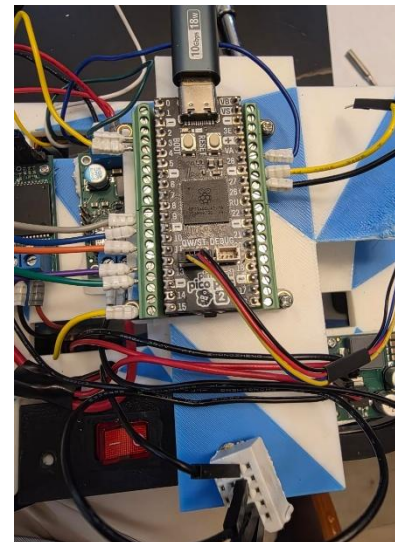


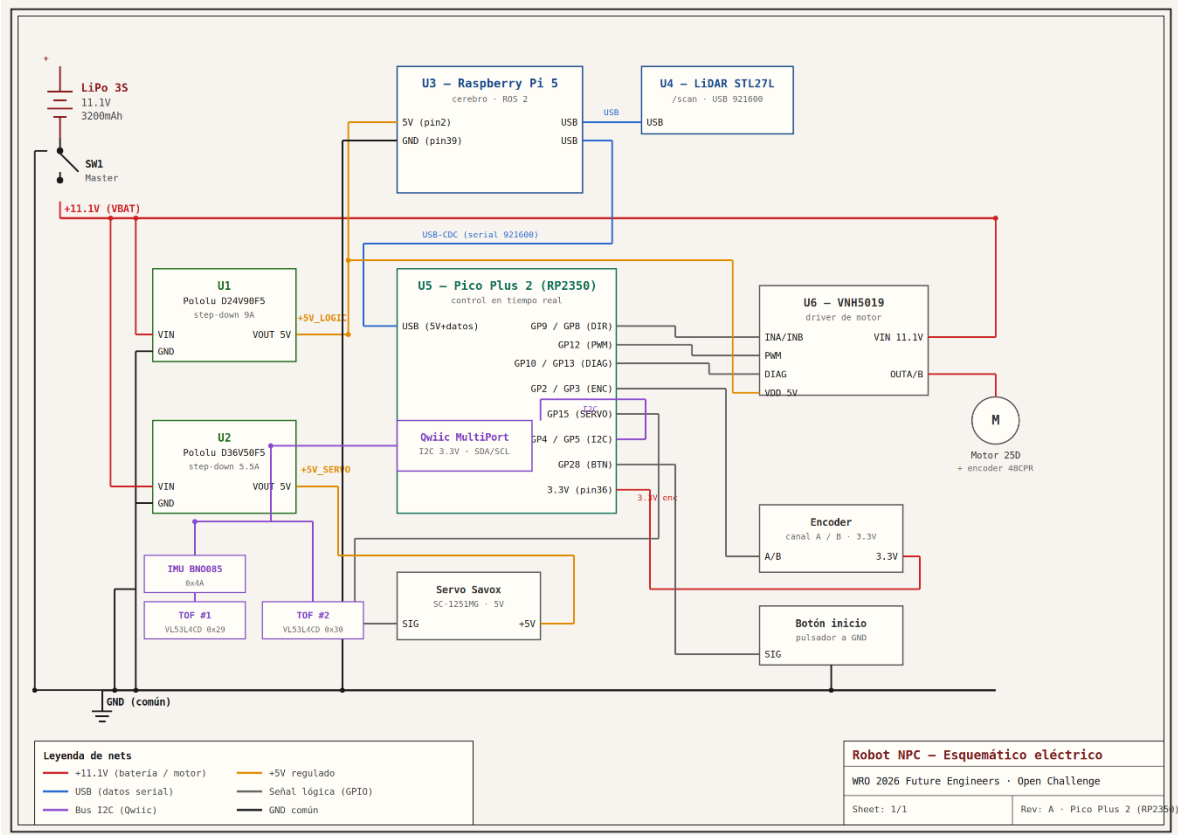
Por la experiencia de los otros equipos, que el microcontrolador era para manejar el volumen de data de los sensores más sencillos, pero con alta frecuencia y por eso el Pico Plus 2 era ideal para manejar al Motor, Servo y el IMU BNO085 y TOF. Si bien el Pico 2 Plus es potente sabíamos que la carga más pesada y la lógica del control era tarea para la Raspberry Pi 5 que con 16 Gb tiene la potencia para manejar la lógica, el LiDAR y la Cámara.

Semana 8 a 9: Diseño Electrónico

A la espera de los componentes, fuimos avanzando en el diseño eléctrico y en la definición de las conexiones de cableado tanto de energía como de conexión entre los distintos componentes. Este proceso fue refinado luego de la llegada de los componentes por se identificaron mejores eficiencias, para evitar cables largos o atravesados, lo cual se explicación más detalle en la Fase 3.

Se prefirió utilizar cables Dupont pero garantizando que fueran 20 AWG y no el tradicional 22 AWG, esto para garantizar un mejor flujo de corriente y evitar conexión débil. Adicionalmente dado que el Pico 2 Plus estaba conectado en un Adafruit Terminal PiCowbell con de sockets y los reguladores de voltaje también usaban sockets, para esas conexiones sustituimos los conectores Dupont por puntas virola o de puntas huecas, que garantizan un mejor ajuste dentro del socket.





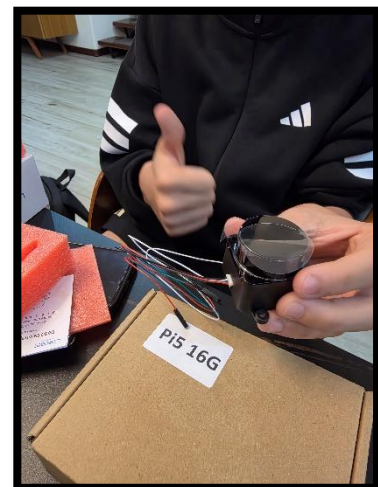
2.3 Fase 3: Construcción y Conexión del Robot “Proyecto Mahoraga” (Semanas 10-14)

	16 al 22 Mar	13 al 29 Mar	30 Mar al 5 Abr	6 al 12 Abr	13 al 19 Abr	20 al 26 Abr	27 Abr al 3 May
	Semana						
Hitos	7	8	9	10	11	12	13
Fase 3: Construcción y Conexión del Robot “Proyecto Mahoraga” (Semanas 10-14)							
Diseño Mecánico Final							
Armado del Robot							
Integración de Electrónica							

Semana 10: Diseño Mecánico Final



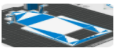
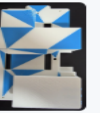
Con la llegada de los componentes, comenzamos a armarlos y revisarlos.

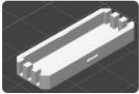
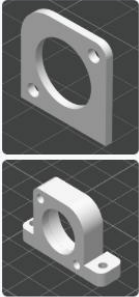


Durante esa semana nos enfocamos por completo en la estructura física del robot, pasando de prototipos básicos a una solución definitiva mediante impresión 3D:

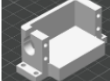
- Diseñamos e imprimimos una versión base del chasis inferior y superior. El objetivo principal de este primer modelo fue evaluar a escala real la distribución espacial de los componentes y verificar que todo encajara físicamente.
- Definimos y aseguramos un compartimiento especial dedicado exclusivamente a la batería, garantizando que quedara fija y no afectara el centro de gravedad del vehículo y fuera fácil de reemplazar sin afectar otros componentes.
- Tras las primeras pruebas, modificamos el chasis para mejorar el campo de visión del LiDAR. Para lograrlo, reubicamos el servo motor, rediseñamos el habitáculo de la batería para que fuera más seguro y organizamos mejor el enrutamiento de los cables. Todo esto nos permitió aprovechar el espacio y mantener un chasis compacto, logrando la agilidad que queríamos.

Todos los archivos de las piezas que modelamos e imprimimos en 3D quedaron organizados en nuestro GitHub dentro de la carpeta **/models**

Pieza	Prototipo	Versión_#1	Versión_#2	Versión_Final	Foto_Real	Comentarios
Chasis de Abajo						El Chasis de Abajo ha tenido varias evoluciones, desde un Prototipo en cartón que hicimos para ver si se ajustaban las piezas. La Versión 1 incluye un agujero para colocar la transmisión, así como el espacio necesario adelante para colocar el LiDAR. La Versión 2 mas refinada, incluye algunos agujeros para tornillos del soporte del Chasis de Arriba, las C Hub incorporadas para la Dirección. La Versión Final además de incorporar mas agujeros para los tornillos de los soportes, también incrementa el espacio para el LiDAR y así lograr un FOV libre de 180 grados y esta impreso con un Infill de 55% para lograr mayor consistencia y se fortalecieron los C Hub, que comenzaron a doblarse en la versión anterior.
Chasis de Arriba						El Chasis de Arriba comenzamos con un Prototipo en cartón para visualizar los componentes a comprar tenían el espacio requerido. La Versión 1 incluye el habitáculo de la batería entre ambos niveles, pero una Versión 2 refinó espacio para los cables, soporte para el suiche de encendido y orificios para los soportes que unen ambos chasis. La Versión Final, refina la colocación del suiche y se crea un subnivel para elevar un poco el RPI 5 y el subnivel encima de la batería para comodar mejor otros compotentes. Finalmente un espacio frontal adicional para centrar bien la cámara, justo encima del LiDAR

Pieza	Modelo Fusion	Versión_#1	Versión_Final	Foto_Real	Comentarios
Sujeta Cables		No Aplica			Sujeta cable Dupont, útil para asegurar que los cables dupont conectados al Servo no se suelten, igual lo usamos para canalizar cables dupont a través del chasis
Soporte de Rolineras		No Aplica			Diseñadas para suetar rolinera que soporta el tren trasero del diferencial. El diseño original llevaba 2 (una de cada lado), pero luego del la Regional 1 de Miranda se decidió incluir 2 adicionales para dar mayor soporte.

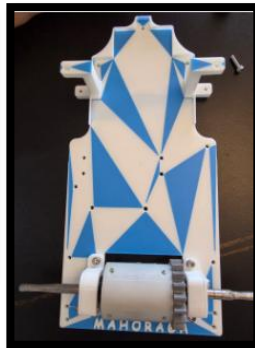
Pieza	Modelo Fusion	Versión_#1	Versión_Final	Foto_Real	Comentarios
Soporte Motor					Un diseño inicial que no ajusto bien el motor y las vibraciones provocaban desplazamientos que movian el ajuste de los engranajes. La Versión Final ajustó esa falla y ademas se colocaron pestañas para tornillos dado que la sujeción por debajo de la V1 hacia complicado su posicionamiento exacto en el chasis
Gear Motor					El engranaje del motor se tuvo que volver hacer luego de la Regional 1 de Miranda, porque el uso comió el hole del engranaje y quedó con poco agarre y hacía que al robot le costara arrancar. Se imprimió nuevamente en Nylon para mayor resistencia al calor de la fricción y al uso. El diseño original se hizo usando una herramienta de diseño de MarketWorld https://makerworld.com/
Gear Diferencial		No Aplica			El diseño original se hizo usando una herramienta de diseño de MarketWorld https://makerworld.com/
Gear Caja de Diferencial		No Aplica			El diseño original se hizo usando una herramienta de diseño de MarketWorld https://makerworld.com/
Espaciador Diferencial		No Aplica			Espaciador en el diferencial para ajustar la caja de engranajes del diferencial a los soportes de la rolineras y que el eje trasero y el diferencial no se muevan lateralmente

Pieza	Modelo Fusion	Versión_#1	Versión_Final	Foto_Real	Comentarios
Soporte Servo					El diseño inicial tenía su atornillado por debajo y dificultaba su ajuste al chasis, además los agujeros a los tornillos quedaron muy flojos por usar un infill muy bajo en la pieza (15%). La Versión Final ajustó esa falla con un Infill de 30% y además se colocaron pestañas para tornillos para mas fácil ajuste al chasis
C Hubs		No Aplica	Descartada, al incluirse en el Chasis en la Versión V1 en adelante		Ideada para sostener las manguetas de la dirección en vez de comprar unas comerciales, sin embargo se incluyeron en el diseño del Chasis V1

Semana 11: Armado del Robot

Resumen del orden recomendado:

1. Imprimir todas las piezas
2. Ensamblar diferencial: insertar engranajes cónicos en Gear Box Diferencial



El diferencial va ajustado en el espacio considerado en el chasis para él, colocando los primeros soportes y los espaciadores hechos a la medida para garantizar que la caja del diferencial no se deslice de lado cuando gire el motor

3. Montar eje trasero KYX con diferencial en el Chasis Abajo ajustando con sus Soportes y Espaciadores. Colocando las Ruedas Traseras de Aluminio de 11 mm de grosor.

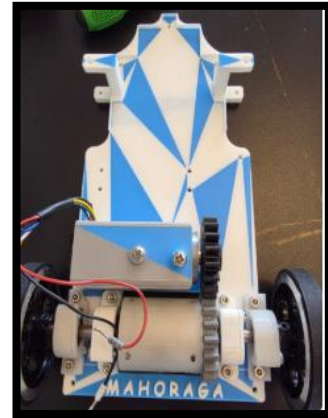
El otro par de soportes se incluyen para evitar que no quede recto la transmisión, como pasaba en la versión anterior del Chasis por el peso de la caja del diferencial. Se ajustan las ruedas y se revisa que queden bien sujetas y a la misma distancia del chasis a cada lado.



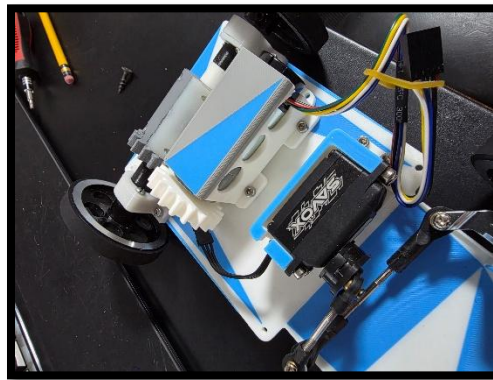
4. Montar Motor Pololu 34:1 con Soporte Motor y Gear Motor con Flange

Los agujeros del chasis garantizan que el soporte del motor quede ubicado donde los engranajes del motor y del diferencial están alineados.

Importante: El engranaje del Motor impreso en Nylon, permite a diferencia del PLA mayor resistencia a la fricción.



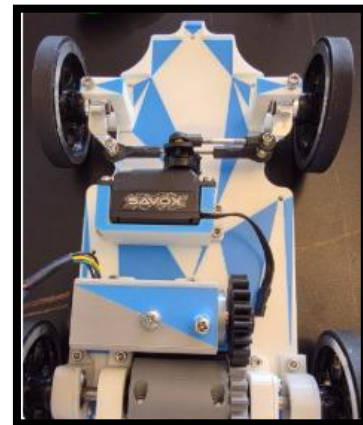
5. Montar servo Savox detrás de las ruedas delanteras en su Soporte Servo VF



6. Conectar manguetas KYX + tie rods M3 + servo saver Tamiya

Las barras se atornillan a las manguetas y estas a las C-Hubs del chasis. Las tierods son ajustables para lograr la alineación con el centro del servo. El soporte del servo se fija al chasis y se atornilla el tierod al Servo Saver.

Se debe ajustar para garantizar la alineación con servo centrado



7. Colocación del LiDAR STL27L

El LiDAR se atornilla en la parte frontal asegurando que quede fijo y garantice un Field of View de 180 grados.



8. Colocar Soportes de Chasis de Arriba

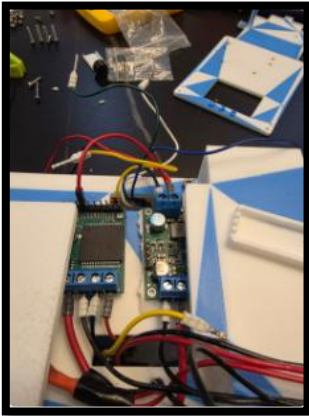
Importante: Los agujeros del chasis superior e inferior están alineados para colocar los soportes de 6cms y así unir ambos chasis, garantizando el espacio con holgura para que los componentes y el subnivel de la batería entren sin problema.



9. Colocación del Chasis de Arriba



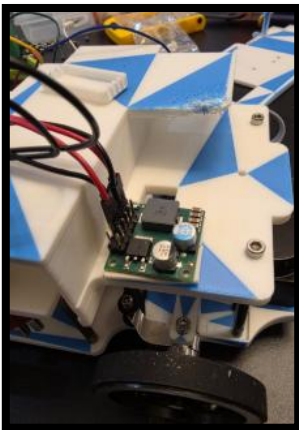
10. Montar VNH5019, reguladores con batería LiPO S3 en el primer piso



Continuamos con la parte eléctrica, colocando el VNH5019 y el Regulador Central que reciben energía de la batería a través del switch de encendido

El detalle del cableado está en la siguiente sección, cable por cable y componente por componente.

11. Colocar y cablear el Regulador del Servo



Arriba en la parte frontal ubicamos el regulador D36V50F5, encargado exclusivamente del servo. Recibe la energía a través del switch y la envía al Servo.

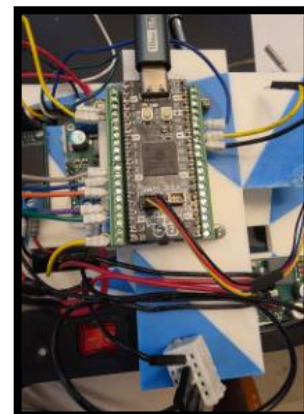
Pasamos los cables por los agujeros del chasis para conectar con el servo que está abajo. Para asegurar los cables Dupont utilizamos un sujetador.



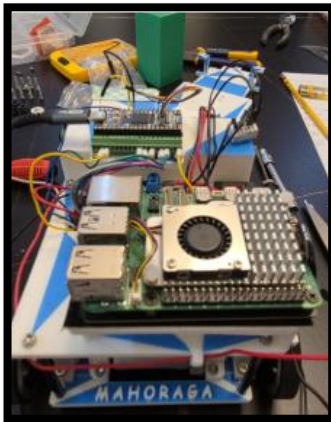
12. Montar el Pimoroni Pico Plus 2 y cablearlo

El Pico Plus 2 va montado encima del Adafruit Terminal PiCowbell for Pico with Pre-Soldered Sockets. Los sockets son de mucha utilidad porque garantizan que los cables no se van a salir porque están atornillados.

Posicionamos la Pico Plus 2, encima del compartimiento de la batería. Queda centrado en el vehículo, lo cual era importante porque es el componente que más conexiones lleva (VNH5019, Encoder, Botón de Arranque, RPI5, IMU y TOF).



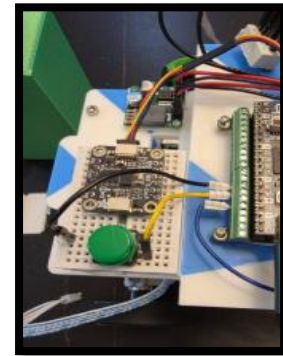
13. Colocación de Raspberry Pi 5



La RPI5 la ubicamos en la parte trasera y sus conexiones principales son con el Pico 2 Plus y el LiDAR, como la comunicación es por USB es importante tener cables muy cercanos a la medida necesaria. Para armar el NVME seguimos el paso a paso de la página oficial de Pimoroni.

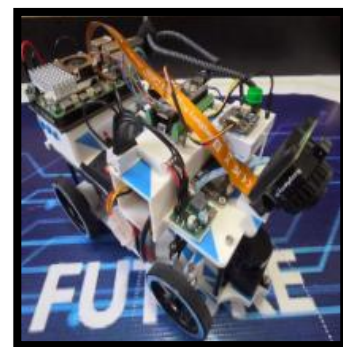
14. Colocación de IMU + Botón de Inicio

Al lado del Pico 2 Plus, colocamos una protoboard y ahí encajamos el Botón de Inicio y el IMU BNO085, asegurándonos que no utilicen las mismas líneas de conexión. Si bien el IMU se conecta por Qwiic los pines pueden interferir.



15. Colocación de Cámara

Colocación de la Cámara en su soporte en el Piso de Arriba con 60 grados de inclinación. Proyecto Mahoraga queda totalmente armado y listo luego de ajustar la programación.



Consideraciones de diseño final:

- Centro de gravedad bajo para estabilidad
- Cámara al frente, elevada para mejor visión
- Batería centrada para balance
- Fácil acceso a electrónica para debugging
- LiDAR abajo y adelante para mejor escaneo de pista

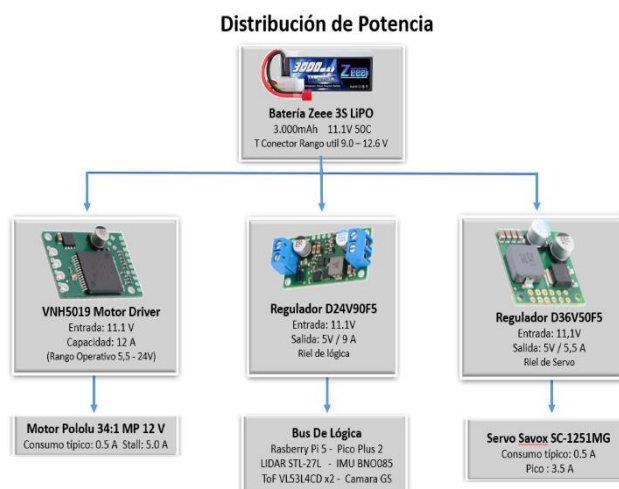
Semana 12: Integración de Electrónica

Conexión de Corriente:

Distribución de corriente entre los dos reguladores de voltaje, decidiéndose dejar el servo con un regulador separado dado que podía generar mayores picos y perturbar la estabilidad de la conexión del Raspberry Pi 5.

Se realizó conexión de la batería Lipo S3 11.1v pasando por el interruptor a:

- 1) Controlador del motor VNH5019
- 2) Pololu 5V, 9A Step-Down Voltage Regulator D24V90F5 y
- 3) 5V, 5.5A Step-Down Voltage Regulator D36V50F5.



Conexión de Cableado de Conexiones:

- Decisión de conectar el Raspberry Pi 5 con el Pico 2 Plus, a través de USB en vez de UART, por facilidad de conexión y simplicidad, pese a su mayor volumen logramos ubicarlo eficientemente.
- Se realizó conexión de comunicación entre Pico 2, Controlador del motor VNH5019 y el encoder del motor.
- Conexión vía Qwiic entre Pico 2 Plus y BNO085 y VL53L4CD. La forma más sencilla de conexión y nos liberó del uso de varios cables Dupont.
- Conexión vía USB entre RPI5 y LiDAR, conexión sugerida por el fabricante.
- Conexión vía cable FPC de 22-pin a 15-pin entre RPI5 y Cámara
- Se realizaron todas las conexiones para tener una tierra común del circuito eléctrico en una mini protoboard.

Para la conexión entre el Raspberry Pi 5 con el Pimoroni Pico 2 Plus y con el LiDAR, utilizamos cables USB a USB-C para transferencia de datos USB 3.1 que garantizan una velocidad de transmisión de 10 Gbps.

Detalle de Cableado

Alimentación principal (LiPo 3S)

Desde	Hacia	Cable
LiPo (+)	VNH5019 VIN	Rojo
LiPo (+)	D24V90F5 VIN	Rojo
LiPo (+)	D36V50F5 VIN	Rojo
LiPo (-)	GND común	Negro

VNH5019 (driver de motor)

Pin VNH5019	Conectado a	Función
VIN	LiPo (+)	Alimentación de potencia (5.5–24 V)
GND (lado derecho)	GND común	Tierra
OUTA	Motor (+) cable rojo	Salida motor canal A
OUTB	Motor (-) cable negro	Salida motor canal B
INA	Pico GP9	Control dirección A
INB	Pico GP8	Control dirección B
PWM	Pico GP12	Modulación velocidad
ENA / DIAGA	Pico GP10	Diagnóstico canal A
ENB / DIAGB	Pico GP13	Diagnóstico canal B
CS	(sin uso)	
VDD	D24V90F5 5V	Alimentación lógica

Encoder del motor

Cable	Color	Conectado a
VCC	Azul	Pico Plus 2 pin 35 (3.3 V)
GND	Verde	Pico Plus 2 pin 3 (GND)
ENCA	Amarillo	Pico Plus 2 GP2
ENCB	Blanco	Pico Plus 2 GP3

Servo Savox SC-1251MG

Cable servo	Color	Conectado a
Signal	Amarillo	Pico Plus 2 GP15
V+	Rojo	D36V50F5 VOut (5 V)
GND	Negro	D36V50F5 GND (uno de los 4)

Bus Qwiic (I²C) Pico → Sensores

Pico Plus 2 Qwiic conectado a SparkFun Qwiic MultiPort

- BNO085 (IMU)
- VL53L4CD #1 (ToF izquierdo)
- VL53L4CD #2 (ToF derecho)

Bus USB (RPI 5)

Periférico	Puerto
Pico Plus 2	USB-A del RPi 5 (cable USB-C 3.1 Gen2 corto, 6 in)
LiDAR STL-27L	USB-A del RPi 5 (vía adaptador CP2102 incluido)

Bus FPC (cámara)

Cable FPC 22-pin 0.5 mm → 15-pin 1 mm, 200 mm, conectado del puerto CSI-2 del RPi 5 al conector de la cámara Global Shutter.

Alimentación de la lógica (D24V90F5 5V)

Desde	Hacia	Pin RPi
D24V90F5 VOut (5 V)	RPi 5 pin 2 (5V)	GPIO header
D24V90F5 GND	RPi 5 pin 39 (GND)	GPIO header
(GND adicional)	RPi 5 pin 6 (GND)	a GND común

Botón de inicio

Pin botón	Conectado a
GND	Pico Plus 2 pin 32 (GND)
Signal	Pico Plus 2 pin 28 (GP) con pull-up interno

2.4 Fase 4: Programación, Pruebas e Iteración (Semanas 12-20)

	13 al 19 Abr	20 al 26 Abr	27 Abr al 3 May	4 al 10 May	11 al 17 May	18 al 24 May	16 al 31 May	1 al 7 Jun	8 al 15 Jun	15 al 22 Jun	20 al 27 Jul	10/8 al 10/9	17-sept
	Semana												
Hitos	11	12	13	14	15	16	17	18	19	20	21	21	21
Fase 4: Programación, Pruebas e Iteración (Semanas 12-20)													
Instalación de Software y Paquetes													
Prueba de Componentes													
Estructura de Código													
Creación de Código del Reto 1													
Pruebas e Iteración para el Reto 1													
Creación de Código del Reto 2													
Pruebas e Iteración para el Reto 2													

Semanas 12 y 13: Instalación de Software y Paquetes

- Instalación en RPI5
 - Ubuntu 24.04
 - ROS 2
 - Paquetes de Lidar, Sensores y Cámara

Setup del Raspberry PI 5

1. Bajar a PC/Laptop el Raspberry Pi Imager desde <https://www.raspberrypi.com/software/>
2. Insertar la MicroSD y correr el Raspberry Pi Imager, seleccionar el Sistema Operativo deseado que es el Ubuntu 24.04 Desktop (64-bit), luego pide la creación de usuario y clave, así como la configuración del WiFi, aunque esto se puede hacer después. Luego comienza a instalar en la Micro SD el Ubuntu 24.04.
3. Cuando termina de grabar, se introduce la Micro SD en la RPI5 y además lo más sencillo es conectarle un teclado, mouse y monitor, utilizando los puertos que la RPI 5 dispone.
4. Al encender la RPI5 se termina la configuración incluido, idioma, teclado, wifi y vuelve a pedir creación de Usuario y Password
5. Configuramos ahora el servidor SSH, para poder acceder a las RPI5 sin necesidad de estar siempre conectado a un monitor, teclado y mouse, sino via wifi desde una laptop. Esto es sumamente importante dado que la RPI va estar en el carro y será más sencillo para hacer pruebas.
6. Para configurar el SSH, se va al Terminal del sistema dentro del RPI y se colocan los comandos para actualización de paquetes y del servidor SSH

```
sudo apt update
```

```
sudo apt install openssh-server -y
```

7. Luego lo activamos para que esté disponible siempre que se use el robot

```
sudo systemctl enable ssh
```

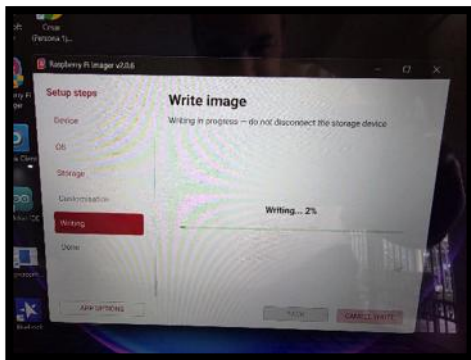
```
sudo systemctl start ssh
```

8. Buscamos el IP del RPI5 en la Red

- `hostname -I` en nuestro caso fue 192.168.68.54

9. En la Laptop ahora se busca el prompt del sistema y ahí se coloca

- `ssh usuario@IP` en nuestro caso `ssh npc@192.168.68.54`
- Pregunta la contraseña y aparece `npc@npc:~$`, ya no que está conectado dentro del laptop si no a la RPI 5. En nuestro caso aparece: `npc@npc:~$`



Agregar los repositorios de ROS 2

```
bash
sudo apt install software-properties-common
sudo add-apt-repository universe
bash
sudo apt update && sudo apt install curl -y
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o
/usr/share/keyrings/ros-archive-keyring.gpg
bash
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-
keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo
$UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Instalar ROS 2 Jazzy

```
bash
sudo apt update
sudo apt upgrade -y
bash
sudo apt install ros-jazzy-ros-base -y
Esto tarda unos minutos.
```

Configurar el entorno

```
bash
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Instalar herramientas adicionales

bash

```
sudo apt install python3-colcon-common-extensions python3-rosdep -y
```

```
sudo rosdep init
```

```
rosdep update
```

- Instalación en Laptop
 - Ubuntu 24.04
 - Setup de Conexión de RPI5 por SSH
 - ROS 2
 - Rviz2
 - Arduino IDE
 - Visual Code

Instalación de Ubuntu y ROS 2

La instalación de Ubuntu y ROS2 en la laptop es similar a la de la RPI. Nosotros usamos una partición de disco distinta para tener el Ubuntu separada totalmente de Windows y utilizamos la versión de Server.

Setup de Conexión de RPI5 por SSH

Ubuntu Server 24.04 ya trae `openssh-server` activo, pero verificar con `sudo systemctl status ssh`. Si no estuviera:

```
sudo apt install openssh-server y sudo systemctl enable --now ssh.
```

Busqueda de IP estable

`hostname -I` en el Pi para conocer la dirección. Lo puedes buscar también en la Red donde se está configurando la dirección MAC.

Para poner conectarte desde la laptop deberás colocar en el Terminal `ssh usuario@192.168.68.x` → contraseña.

RViz2 (Visualización de Sensores)

Para ver lo que el Lidar detecta o lo que la cámara. ROS 2 permite que la RPI envíe solo los datos procesados a la laptop. Tú abres un programa llamado **RViz2** en tu laptop y ahí verás el mapa y los obstáculos en 3D sin sobrecargar la Raspberry Pi.

Este programa debe correr directamente en la laptop (no por SSH).

- **Su función:** Como ROS 2 es un sistema distribuido, la Raspberry Pi "transmite" los datos del Lidar y del IMU por el Wi-Fi, y **RViz2 en tu laptop** los recibe y los dibuja.

Para instalarlo en la laptop luego de tener Ubuntu y ROS2 instalados:

```
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp  
export ROS_DOMAIN_ID=<el mismo número en Pi y laptop>
```

Arduino IDE

1. Descargar el archivo:

- Entra al sitio web oficial de Arduino IDE
<https://support.arduino.cc/hc/en-us/articles/360019833020-Download-and-install-Arduino-IDE>
- Busca la sección para **Linux** y descarga la versión de 64 bits en formato **Applimage**.

2. Ejecutar el programa:

- Haz doble clic en el archivo .Applimage o haz clic derecho y selecciona **Ejecutar** para abrir el IDE de Arduino.

3. Configurar permisos para la placa USB:

- Para que tu computadora reconozca la placa Arduino por el puerto serie sin usar comandos de administrador, añade tu usuario al grupo dialout:

```
bash  
sudo adduser $USER dialout
```

Visual Code

1. Abre tu navegador y ve a la Página oficial de descargas de VS Code.
<https://code.visualstudio.com/docs/setup/linux>
2. Haz clic en el botón **.deb** (para Debian/Ubuntu) para descargar el instalador.
3. Abre la terminal y dirígete a tu carpeta de descargas:

```
bash  
cd ~/Descargas
```

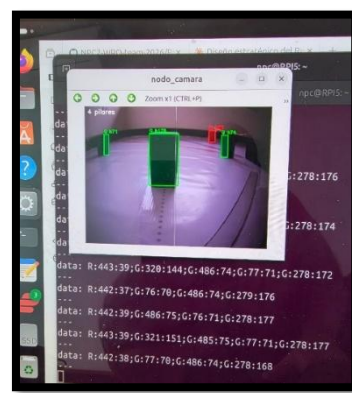
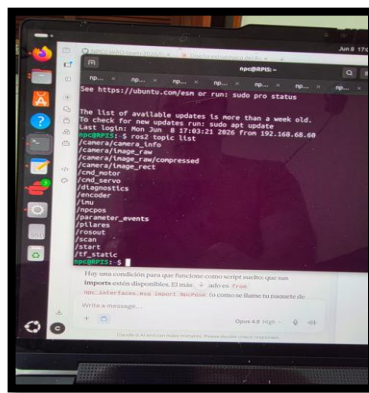
Instala el paquete ejecutando el siguiente comando:

```
bash  
sudo apt install ./code_*.deb
```

Semana 14: Prueba de Componentes

- Pruebas de Conexión RPI5-Laptop a través de protocolo SSH conectando ambos dispositivos a la misma red y la conexión efectiva abriendo un terminal de RPI5 desde la Laptop.

- Pruebas de Conexión RPI5-Pico 2 Plus, conectada la Pico 2 a el puerto /dev/ttyACM0, utilizamos pyserial para confirmar la comunicación entre ambos controladores.
- Pruebas de LiDAR, conectando a la RPI 5 bajo el puerto /dev/ttyUSB0 y ejecutando en la terminal el nodo de ROS 2 correspondiente al LiDAR, encargado de leer los datos de distancia por del tópico de tipo /scan. Finalmente, se lanza RViz2 en el entorno de la Laptop para visualizar la nube de puntos marcada por el LiDAR
- Pruebas de Cámara. Vista y configuración de colores y poder evidenciar los tópicos



- Realización de Pruebas de IMU BNO085, Motor, Encoder y Servo

Realizamos un Test integral para ver que todo funcione sin problema y en el sentido correcto.

El Test está en la carpeta de SRC (Test de Robot.py, pero en todo caso so queríamos mover el motor 2 segundos en cada dirección mover el servo y la medición de las variables del IU y del encoder.

Luego de varias pruebas y anotar algunas convenciones de derecha e izquierda, pasamos al siguiente punto. Estas pruebas las repetimos varias veces en alguna ocasión cuando el código no lográbamos hacer lo que queríamos y así descartábamos algún mal funcionamiento.

Semana 14 y 15 Estructura de Código

Dado que el Pimoroni Pico 2 Plus, como microcontrolador, lo designamos encargado de la gestión en tiempo real de los motores, sensores Encoder, TOF e IMU, debe por una parte proveer de toda la lectura de información al Raspberry Pi 5, que junto con la información que el recopila de la Cámara y el LiDAR pueda tomar decisión en base al código o estrategia suministrada. Esas decisiones por otra parte deben ser transmitidas al Raspberry Pico 2 Plus para accionar.

En base a lo anterior decidimos elaborar un código en Arduino .ino que realiza las mediciones de los sensores y transmite al Raspberry Pi 5. Dado que decidimos trabajar con ROS2 y manejarlos a través de nodos y tópicos, era necesario un código bridge en Python que hicieran legible el código ejecutado en Arduino IDE tanto de salida al RPi5 como de entrada de las instrucciones a los motores.

Finalmente, el control de las instrucciones y estrategia de abordaje del reto debía estar en Python desde donde recibía la información y mandaba al nodo bridge para comunicarse con el Pico 2 Plus, pero también recibía la información de los tópicos creados para la información de la Camara y el Lidar.

```
src/
├── firmware/
│   └── pico_firmware.ino          # Firmware C++ del Pico (Arduino)
├── npc_bot/
│   ├── pico_bridge_node.py        # Puente USB ↔ topics ROS 2
│   ├── control_node.py           # Reto 1: Open Challenge
│   ├── control_node_reto2.py      # Reto 2: Obstacle Challenge
│   ├── nodo_camara.py             # Visión (solo Reto 2)
│   └── calibrador_hsv.py          # Calibración HSV interactiva
├── npc_interfaces/
│   └── msg/NpcPose.msg            # Mensaje custom {x,y,θ,acc_yaw}
├── launch/
│   ├── npc_bot.launch.py         # Lanza Reto 1 completo
│   └── base_reto2.launch.py      # Lanza Reto 2 completo
└── systemd/
    ├── npc_bot.service           # Arranque Automatico Reto 1
    └── npc_bot_reto2.service      # Arranque Automatico Reto 2
```

Firmware del Pico 2

El Pico Plus 2 ejecuta un firmware basado en Arduino (pico_firmware.ino) que maneja todas las operaciones de tiempo real de las que la RPi5 no debe encargarse.

- Lectura del IMU a 100 Hz vía Qwiic I2C (Adafruit BNO085, con fusión sensorial 9-DOF onboard)
- Conteo del encoder vía interrupción por hardware en GPIO 2 (Canal A) y lectura digital en GPIO 3 (Canal B), cuadratura de un solo flanco
- Integración de odometría a 100 Hz (calcula Δx , Δy , $\Delta \theta$ y los acumula)
- Generación de PWM del motor a 20 kHz (sobre el rango audible) vía driver Pololu VNH5019
- Generación de PWM del servo a 50 Hz para el servo de dirección Savox SC-1251MG
- Lectura del botón de arranque en GPIO 28 (INPUT_PULLUP, activo LOW)
- Publicación de telemetría a la Pi5 a 50 Hz vía USB serial

Protocolo Serial de Comunicación RPi5 y Pico2

Pico → **Pi5** (50 Hz cada uno):

POS <x, y, theta, acc_yaw> → Odometría IMU <yaw, qx, qy, qz> → Cuaternión del IMU
ENC < ticks > → Conteo del encoder BTN → Botón presionado (evento) BOOT,... OK,...
ERR,... → Mensajes de estado
Pi5 → **Pico** (a demanda):

M < velocidad_ms > → Velocidad del motor en m/s S < direccion > → Servo, rango [-1.0, +1.0] STOP → Parada de emergencia

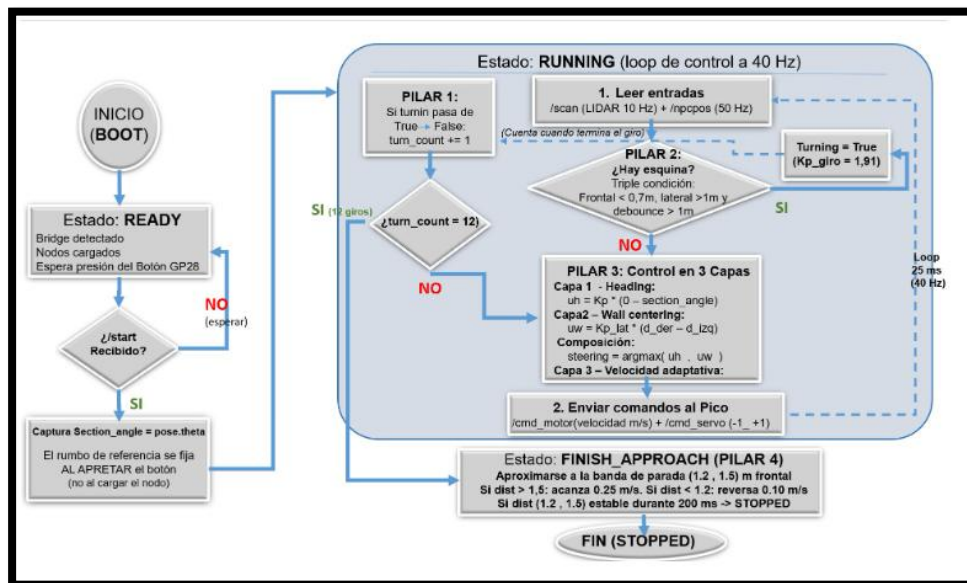
Tópicos

Se desarrolló un esquema de comunicación basado en tópicos, estableciendo los nodos suscriptores y publicador fundamental para la comunicación en ROS2.

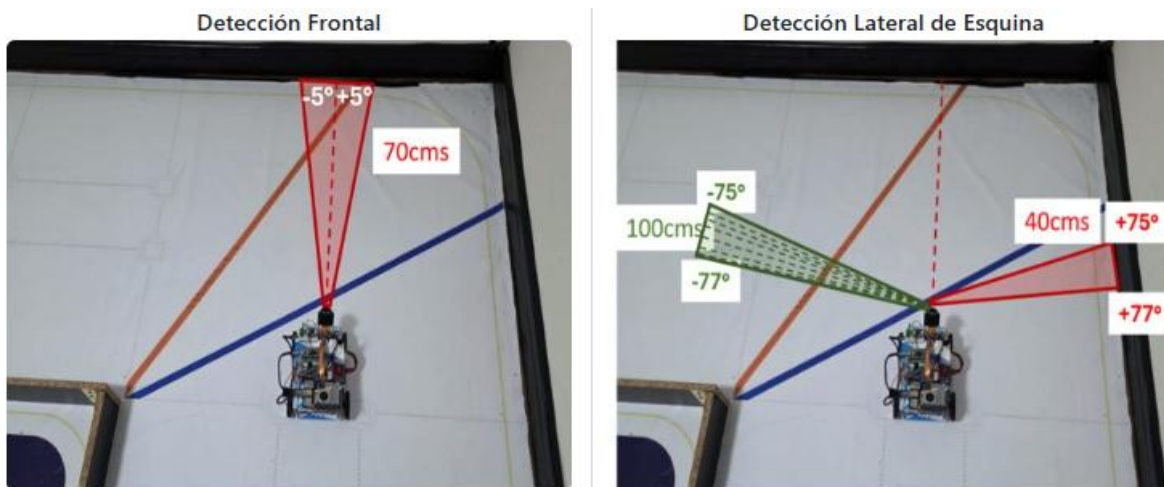
Topico	Tipo	Publicador	Suscriptor	Descripción
/scan	sensor_msgs/LaserScan	Driver LiDAR	control_node	Scan LiDAR 360° @10 Hz
/npcpos	npc_interfaces/NpcPose	pico_bridge	control_node	Pose del robot (x, y, theta, acc_yaw) @50 Hz
/start	std_msgs/Empty	pico_bridge	control_node	Señal de inicio (botón)
/cmd_motor	std_msgs/Float32	control_node	pico_bridge	Velocidad del motor en m/s
/cmd_servo	std_msgs/Float32	control_node	pico_bridge	Dirección en [-1.0, +1.0]
/imu	sensor_msgs/Imu	pico_bridge	(debug)	Datos del IMU filtrados
/encoder	std_msgs/Int32	pico_bridge	(debug)	Ticks crudos del encoder

Semana 16 y 17 Creación de Código del Reto 1

La estrategia para el reto 1 (sin obstáculos) no visionábamos que fuera muy distinta a la que utilizamos en 2025. El robot debe detectar cuándo ha llegado a una esquina, decidir hacia qué lado girar, y en qué sentido (CW o CCW) seguirá toda la carrera. La diferencia es que en 2025 lo hicimos con ultrasónicos y ahora contamos con un LiDAR, esto nos permitió hacer la detección de una forma más robusta.



Al encontrarnos a 0,7 metros de la pared central, el LiDAR revisa los laterales en busca de aperturas (lecturas mayores a 1m). Sin embargo, no dependimos de una medida única frontal del LiDAR y 2 laterales, le pedimos la mediana de $\pm 5^\circ$ de la medición frontal y 2° de mediciones laterales en vez de solo una medición a $+75^\circ$ y -75° . En el caso de nuestro LiDAR que nos da 2.160 mediciones en cada vuelta, esto representa 6 mediciones por cada grado, permitiéndonos tener 60 mediciones frontales (10×6) y 12 en cada lateral (2×6), dando más robustez y evitar falsas señales.



Luego de la detección de la apertura se activa el giro y lo mas importante es determinar hasta cuando debe girar el robot. El año pasado estuvimos muy limitados por que tuvimos problema para hacer funcionar el IMU. Este año con el BNO085 conectado a la Pico Plus 2 tuvimos resuelto, siendo la función del IMU indicar cuando habíamos logrado alcanzar 90 grados adicionales, por lo que no importaba si el robot llegaba con cierto desvío a la esquina, porque el IMU le iba a indicar al robot que debía girar hasta lograr la posición objetivo ($+90$ grados en cada esquina).

No obstante, para hacer la tarea mas sencilla, mantuvimos un sistema de ajuste para que mientras el robot no estuviera girando porque estaba en una recta, tratará de mantenerse lo mas centrado posible, para eso, usamos mediciones laterales de cada lado del LiDAR que mandan a realizar ajustes en la dirección dependiendo de la diferencia entre la distancia a la pared derecha – la distancia a la pared izquierda.

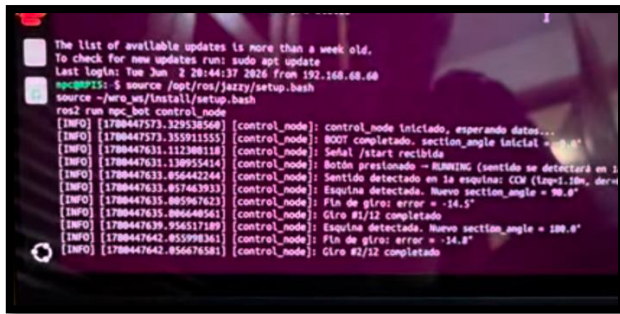
```
pos = dist_der - dist_izq
str_lateral = clamp(pos * KP_LATERAL, -0.5, +0.5)
```

Finalmente, para garantizar culminar en el segmento que comenzó el robot luego de las 3 vueltas, se configuró una estrategia que después de 12 giros, el robot continuara adelante a una velocidad reducida y parara cuando estuviera a 1,5 m de la pared frontal. Lo que garantizaba parar en el centro de la sección.

Para mayor detalle de las líneas de comando principales se puede ver el **segmento 6 del Readme** y para ver el código con anotaciones se pueden obtener los archivos en la carpeta **/SRC** del Github, estando el código principal del reto 1 en el archivo **control_node.py**

Semana 17 a 19: Pruebas e Iteración para el Reto 1

Desde el comienzo los resultados de las pruebas fueron bastante positivos, con varias iteraciones sobre todo en ajustar velocidad y otros parámetros como agresividad del giro, distancia ideal para activar la detección de apertura, considerando que las condiciones del centro de la pista pudiera ser variables.



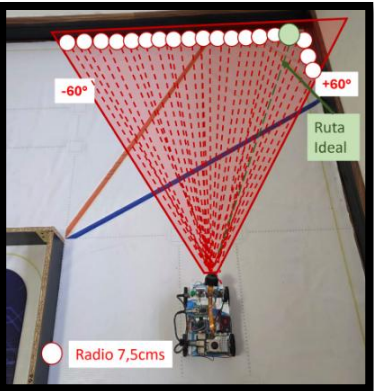
Al final como equipo dimos mas peso a ser confiables que a lograr menores tiempo, por lo que, pese a que en ocasiones lográbamos bajar los tiempos de 70 segundos a 64 segundos y finalmente a 55 seg, este último no era totalmente libre de errores.

Métricas de Desempeño Reto 1.

Métrica	Junio	Julio	Ago-Sept
Detección de sentido (CW o CCW)	100% correcta con el enfoque de detección en la primera esquina y luego de ajustes.	Igual	Igual
Precisión del conteo de giros	12/12 giros contados correctamente, no se generan dobles giros por restricciones colocadas para ello, aprendidas en WRO 2025	Igual	Igual
Tasa de completación de vueltas	Luego de ajustes, Estable con múltiples corridas exitosas de 3 vueltas, 1 solo choque por fallo en lectura de apertura, corregido	100%	100%, excepto en pruebas con mayor velocidad 0,6 m/s donde la efectividad fue 60%
Precisión de posición de parada	Consistentemente dentro de la banda 1,2 a 1,5m	Igual	Dentro de la banda de 1,4 a 1,7m al probar con más velocidad
Posición lateral durante rectas	< 5 cm de desviación del centro del corredor, mucho más robusto con el control dual Heading y Centrado	Igual	Igual
Heading después de completar un giro	Dentro de 5° del objetivo	Igual	Igual
Tiempos Promedios en Pruebas	70 segundos	Se lograron tiempos de 64 seg con mayor velocidad 0,5 m/s, pero las curvas eran mas abiertas, por lo que decidimos mantener en 0,4 m/s, especialmente cuando el centro de la pista era grande 1,8m x 1,8m	Igual, logrando 55seg en algunas pruebas de mas velocidad 0,6 m/s

Semana 18 a 19: Creación de Código del Reto 2

Fue nuestro dolor de cabeza de 2025 y nuestro gran reto para 2026, con la confianza que tenemos un robot más robusto y tecnológicamente avanzado, además de contar con 1 año más de experiencia. El sistema de navegación del reto 1 no funcionaría en el reto 2, la detección de aperturas podía estar condicionado a que un obstáculo limitara la detección de una apertura.



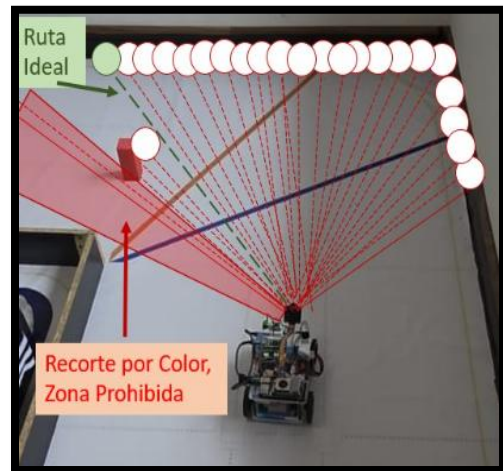
En nuestra investigación vimos que muchos equipos se valían de mapear el camino y generar un mapa de la pista conforme avanzaban. Sin embargo, para nosotros la vía correcta era tener un robot que fuera de manera autónomamente así la pista cambiara, es decir, que incluso pudiera navegar fuera de la pista, tal como lo hace un humano con su vista, donde de alguna manera en un camino buscas la apertura que se va generando o visto de otra manera la ruta mas profunda sin interrupción.

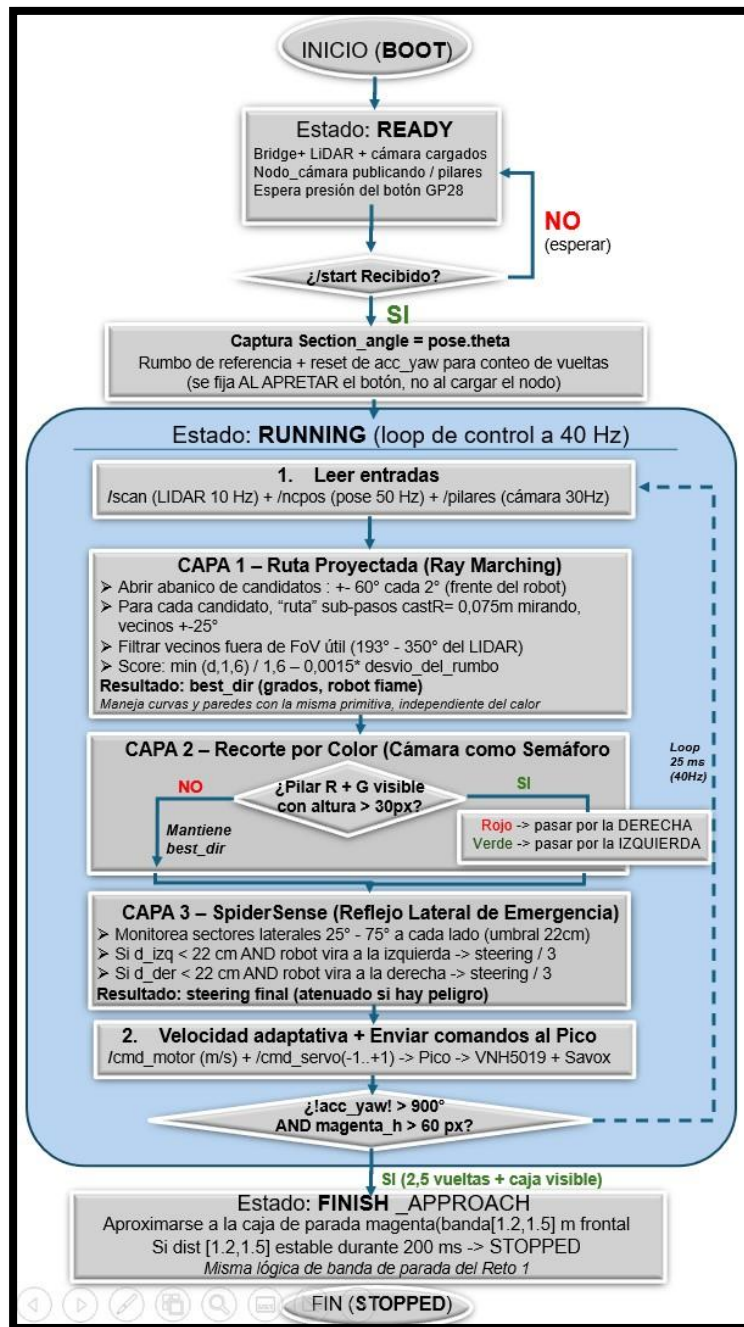
Para traducir esto al robot, el LiDAR debía reportar un abanico de direcciones candidatas frente al robot (de -60° a $+60^\circ$ en intervalos de 2° - 60 lecturas) y cada una se convierte en una "Ruta Viable", entonces para cada una se calcula un "score" para determinar cual es la trayectoria ideal o best_dir. Ese score combina 2 cosas, 1) Cuán lejos puede ir el robot en esa dirección sin chocar con algo y 2) Que tan desviada está esa dirección respecto al rumbo objetivo de la pista. La dirección ganadora es la propuesta inicial de hacia dónde dirigirse.

Adicionalmente para cada una de las 60 rutas candidatas, se aplica el principio de "ray marching" donde no solo se observa el punto candidato, sino que también se mide un radio de 7,5 cm de cada lado para cerciorarnos que el robot pueda caber por esa ruta, si en la ruta candidata se choca con algún objeto al ver el ray marching, entonces esa ruta es descartada o su distancia máxima se limita hasta donde si cabe el ray marching.

Esto garantizaría que, sin obstáculos en la pista, el robot siempre seguirá la ruta más profunda y pudiera hacer el reto 1 sin problema, pero con una lógica de navegación diferente, no hay esquinas, solo se sigue la ruta mas profunda.

Para poder manejar los obstáculos la lógica que se nos ocurrió fue que el robot siguiera buscando la ruta más profunda, pero en presencia de un obstáculo, limitara las rutas a las que estaban por el lado que debe pasar el robot, es decir, si el pilar era rojo, todas las rutas a la izquierda del pilar estuvieran eliminadas de las candidatas y el robot solo podía elegir entre aquellas que garantizaba que pasaban por el lado correcto. Al pasar el pilar, se eliminaba la restricción hasta la detección del nuevo pilar.



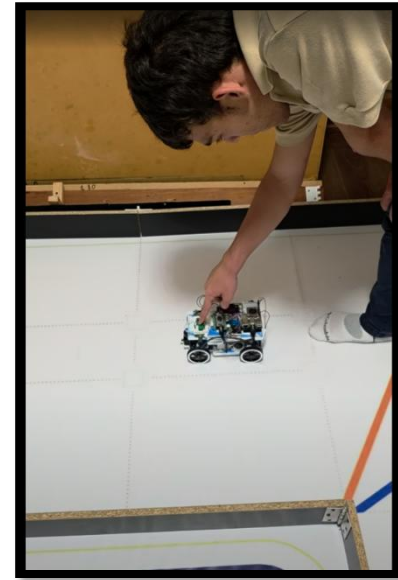


Para mayor detalle de las líneas de comando principales se puede ver el **segmento 6 del Readme** y para ver el código con anotaciones se pueden obtener los archivos en la carpeta **/SRC** del Github, estando el código principal del reto 1 en el archivo **control_node_reto2.py**

Semana 19 a 21: Pruebas e Iteración para el Reto 2

Si bien pensamos que la estrategia para el Reto 2 es la correcta, lograr la combinación ideal de los valores de los parámetros ha sido una labor muy iterativa. Hemos tratado de balancear entre el nivel de velocidad correcto para dar el mayor tiempo de detección temprana a la cámara y al robot de capacidad de respuesta para buscar la ruta ideal que nos lleve a la evasión de los pilares. No obstante, lo pronunciado del giro y el tamaño del Ray Marching para evitar choques laterales del robot ha sido el verdadero reto.

A esto se incorpora otro elemento que es la tonalidad cambiante de los colores dependiendo del nivel de luz y si este es natural o de lámparas, así como que puede cambiar dependiendo en la sección de la pista que se encuentre el robot. Para esto hemos tratado una combinación de códigos para cada color lo suficiente amplia para que no sea tan sensible, sin embargo, estamos consientes que ajustes el día de la competencia, ya en pista serán necesarios.



A continuación, reportamos los avances con la métricas de desempeño del Reto con Obstáculos.

Métrica	Junio	Julio	Ago-Sept
Detección de Ruta Ideal)	100% correcta cuando se practica sin obstáculos (12 rondas de 3 vueltas c/u). Con obstáculos (200 obstáculos con promedio de 5 por vuelta de 40 vueltas). En el 10% de los casos la evasión ha causado un desvío incorrecto (21/200)	Con obstáculos (357 obstáculos con promedio de 7 por vuelta de 51 vueltas). En el 10% de los casos la evasión ha causado un desvío incorrecto (36/350)	420 obstáculos con promedio de 7 por vuelta de 60 vueltas. En el 6% de los casos la evasión ha causado un desvío incorrecto (25/420)
Tasa de Detección de Obstáculos	96% de obstáculos detectados correctamente (192/200 obstáculos). Dificultad de detección cuando vehículo se desvía mucho y cámara no detecta por el FoV	98% de obstáculos detectados correctamente	90% de obstáculos detectados correctamente, se practicó con ubicación de obstáculos compleja
Tasa de Evasión de Obstáculos	80% de obstáculos detectados correctamente (160/200 obstáculos). 4% por no detección, 10% por falta de espacio para evadir y 6% por chocar el obstáculo	85%	89%
Tasa de completación de vueltas	40% de Éxito de vuelta completa con evasión (16 de 40 vueltas). Ha mejorado desde que redujimos velocidad.	50%	90% aunque mueve algún pilar
Precisión de posición de parada	0% Se está deteniendo un giro antes por detección errónea del magenta. Identificado el problema pero dando prioridad a otros ajustes	Igual	40%, aun con problemas con el magenta
Tiempos Promedios en Pruebas	32 segundos por Vuelta	35 seg por vuelta	38 seg por Vuelta, hemos logrado mas eficiencia con menor velocidad